

## TP 3 — Construire sa boîte à outils de fonctions

Semaine 3 ■ Fichiers dans NSI/TP03/ ■ 2 h

### Objectifs

- Écrire des fonctions avec paramètres et `return`, puis les **appeler**
- Vérifier chaque fonction avec des appels de test
- Réutiliser des fonctions pour en écrire d'autres

**Méthode de la séance :** pour chaque fonction, écrire sous sa définition deux ou trois lignes de vérification, par exemple : `print(volume_pave(2, 3, 4), "attendu : 24")`. (Au ch. 8 on utilisera `assert` pour automatiser cela.)

### Partie A — Gammes — `outils.py`

(40 min)

Écrire et tester :

1. `volume_pave(l, L, h)` — volume d'un pavé droit
2. `celsius_vers_fahrenheit(t)` —  $F = 1,8 \times C + 32$
3. `distance(xa, ya, xb, yb)` — distance entre deux points (racine carrée : `x ** 0.5`)
4. `heures_minutes(minutes)` — renvoie **deux valeurs** : heures et minutes restantes (`heures_minutes(130)` → 2, 10)
5. `est_valide(note)` — True si  $0 \leq \text{note} \leq 20$

### Partie B — Fonctions booléennes et composition — `annees.py`

(30 min)

1. `est_bissextile(annee)` — reprendre la règle de la semaine 2, mais en fonction qui **renvoie** un booléen.
2. `nb_jours(annee)` — renvoie 365 ou 366 **en appelant** `est_bissextile`.
3. `nb_jours_pperiode(a1, a2)` — nombre total de jours entre les années `a1` et `a2` incluses (boucle qui **accumule** des appels à `nb_jours`).
4. Vérifier : `nb_jours_pperiode(2000, 2003)` → 1461.

### Partie C — Un vrai découpage — `des.py`

(40 min)

On veut simuler le petit jeu suivant : on lance deux dés ; si la somme est 7 ou 11 on gagne, si elle vaut 2, 3 ou 12 on perd, sinon c'est nul.

1. `lancer_de()` — renvoie un entier aléatoire entre 1 et 6 (fonction **sans paramètre**).
2. `resultat(d1, d2)` — renvoie la chaîne **"gagné"**, **"perdu"** ou **"nul"** selon la règle.
3. Programme principal : lancer deux dés, afficher les dés et le résultat.
4. ★ `statistiques(n)` — joue `n` parties et renvoie le nombre de parties gagnées. Estimer la probabilité de gagner sur 10 000 parties (théorie :  $8/36 \approx 0,222$ ).

### Partie D — Défi ★★

(10 min)

`pgcd(a, b)` — le plus grand commun diviseur par l'algorithme d'Euclide : tant que `b` n'est pas nul, remplacer `(a, b)` par `(b, a % b)` ; renvoyer `a`. Vérifier : `pgcd(48, 36)` → 12, `pgcd(17, 5)` → 1.